Defesa de Viabilidade — Câmara Climatizada (Plataforma SAGA SENAI)

Título do Projeto

Modernização e Supervisão Inteligente para Cabine Climatizada de Ensaios Térmicos

---

1. Qual é a demanda da empresa?

A empresa realiza testes térmicos em dispositivos eletrônicos. Ela coloca até **50 aparelhos** dentro de uma cabine climatizada, liga tudo e espera horas enquanto a câmara esquenta e esfria em ciclos programados.

O problema é que, durante esse tempo todo, **ninguém sabe se um dos aparelhos parou de funcionar**. Se um dispositivo queima no meio do teste, o operador só descobre no final — às vezes depois de 8 horas. Aí não dá para saber **qual** aparelho falhou, **quando** falhou, e o teste inteiro pode ser descartado.

A empresa quer uma solução que:

- Controle a temperatura automaticamente (sem depender de alguém ajustando manualmente).
- **Monitore cada aparelho individualmente** e avise imediatamente se algum parar de funcionar.
- Registre tudo com data e hora, para ter um histórico confiável.
- Permita acompanhar o teste à distância, pelo computador ou celular.

---

2. Por que escolhemos essa demanda?

Escolhemos essa demanda porque ela encaixa perfeitamente no que aprendemos no curso de **Eletrotécnica**:

- **Comandos elétricos:** O projeto usa relés, botão de emergência e selo elétrico — exatamente o que estudamos em aula. A segurança é feita em hardware: se o programa travar, a máquina desliga sozinha.
- **Instalações elétricas:** Precisamos dimensionar fios, disjuntores, fusíveis e fontes de alimentação. O painel tem 3 trilhos DIN com separação entre potência e sinal, como num quadro industrial de verdade.
- **Instrumentação:** Usamos sensores de temperatura e de corrente elétrica. O sensor de corrente é o que permite saber se cada aparelho está funcionando ou não.
- **Automação:** O controle de temperatura funciona em **malha fechada com PID** — o sistema mede, compara com o valor desejado e ajusta sozinho (explicamos o que é PID na seção 10).
- **Redes industriais:** Os sensores de corrente se comunicam por **RS-485**, que é o mesmo barramento usado nas fábricas (explicamos como funciona na seção 11). Isso prova que o projeto não é só escolar: ele usa a mesma tecnologia da indústria.

Em resumo: não é um projeto só de programação. É um projeto de **eletrotécnica com programação**, que é exatamente o perfil do nosso curso.

---

3. Como funciona a nossa solução?

A ideia é simples: colocamos um **sensor de corrente** em cada posição de teste. Se o aparelho está funcionando, passa corrente. Se parou, a corrente cai para zero. Quando isso acontece, o sistema avisa na hora.

Na maquete (o que vamos construir):

- Em vez de uma cabine industrial gigante, usamos uma **caixinha de acrílico isolada** com pastilhas Peltier para esfriar (aquelas "pedrinhas" que transferem calor de um lado pro outro) e um **aquecedor PTC** para esquentar (uma peça cerâmica que gera calor quando ligada na corrente). Ela é pequena, mas funciona igualzinho: esfria, esquenta e controla a temperatura.
- No lugar dos 50 aparelhos, usamos **2 lâmpadas automotivas** como carga de teste. Se a lâmpada queima ou se puxamos o fusível, o sensor detecta que a corrente sumiu e dispara o alarme.
- Usamos **módulos de sensor de corrente com furo de passagem** (como a linha WCS2702). Assim, não precisamos cortar o fio de alimentação para medir a corrente contínua (DC) das lâmpadas — o fio só passa por dentro do furo. *(Nota: se os aparelhos testados pela empresa funcionarem na tomada comum, em Corrente Alternada - AC, o projeto continua idêntico. Nós só trocamos esse módulo por um TC miniatura de corrente alternada).*
- O painel de comando é de verdade: trilho DIN, canaletas, relés, botão de emergência, sinaleiros — tudo como num painel industrial.

O que acontece passo a passo:

1. O operador configura o teste na **IHM** (a tela de interface do painel, com toque na tela): escolhe a temperatura desejada e o tempo do ensaio.

2. O Arduino controla a temperatura automaticamente em malha fechada PID: mede a temperatura, compara com o que o operador pediu, e liga a Peltier (se precisa esfriar) ou o PTC (se precisa esquentar).
3. Os sensores de corrente monitoram as lâmpadas o tempo todo. O sinal vai por RS-485 até o ESP32, que fica perguntando "como estão as correntes?" a cada segundo.
4. Se uma lâmpada apaga, o sistema identifica **qual** posição falhou e **quando**, grava no cartão de memória e manda um aviso pelo Wi-Fi.
5. O operador pode acompanhar tudo pelo computador (dashboard), sem precisar ficar na frente do painel.

---

4. É viável? Dá pra fazer?

Sim, e por 3 motivos:

1. **Custo baixo:** Todos os componentes são vendidos no Brasil e custam pouco. O protótipo inteiro (maquete + painel + câmara + sensores) fica abaixo de **R$ 2.000,00**.

2. **Componentes prontos:** Não estamos inventando nada do zero. Usamos Arduino, ESP32, sensores e módulos que já existem prontos no mercado. É só montar e programar.

3. **Construção por etapas:** O projeto é dividido em 5 camadas (Fundamentos → Maquete → Painel → Elétrica → Integração). Se der problema numa etapa, não perde o que já fez. Dá para testar cada pedaço separado antes de juntar tudo.

---

5. E na fábrica real? Quanto custa implementar de verdade?

A fábrica **já tem o Arduino** controlando a câmara. O que falta é o sistema de monitoramento dos 50 dispositivos. Não dá para ligar 50 sensores direto no ESP32 — seriam fios demais e o ESP32 não tem 50 entradas. A solução é usar **módulos concentradores RS-485**.

Como funciona na indústria:

- Cada grupo de **8 sensores** é ligado a um **módulo ADC de 8 canais** (ADC significa *Analog-to-Digital Converter*, ou Conversor Analógico-Digital. É um "tradutor" em formato de chip que lê o sinal analógico dos 8 sensores e transforma em um número digital que o microcontrolador entende).
- Esses módulos ficam espalhados no quadro de disjuntores, **perto dos sensores** (fios curtos, sem ruído).
- Todos os módulos são ligados entre si por apenas **2 fios** (o barramento RS-485), um atrás do outro, como uma corrente.
- Na ponta dessa corrente, o ESP32 fica perguntando para cada módulo: "módulo 1, como estão seus 8 sensores?", "módulo 2, como estão os seus?" — e assim por diante.
- Com **7 módulos de 8 canais** dá para monitorar os 50 dispositivos (7 × 8 = 56 posições).

Estimativa de custo para a fábrica (50 posições):

Item	Quantidade	Custo unitário	Subtotal
Sensor de corrente tipo "alicate" (TC Split-Core, não precisa cortar o fio)	50 unidades	R$ 30 a R$ 50	R$ 1.500 a R$ 2.500
Módulo ADC 8 canais Modbus RS-485 (concentrador)	7 unidades	R$ 180 a R$ 300	R$ 1.260 a R$ 2.100
ESP32 industrial com RS-485 e Ethernet (tipo DNLCB30 ou ESP32-P4)	1 unidade	R$ 150 a R$ 300	R$ 150 a R$ 300
Cabo par trançado blindado RS-485 (~50 a 100 m)	1 rolo	R$ 150 a R$ 300	R$ 150 a R$ 300
Resistores de terminação 120 Ω, bornes, conectores	—	—	R$ 100 a R$ 200
Software do dashboard (Grafana / Node-RED)	—	**Gratuito**	R$ 0
TOTAL ESTIMADO			**R$ 3.300 a R$ 5.400**

> Esse valor é para monitorar **todos os 50 dispositivos de uma vez**. Comparado com o custo de descartar lotes inteiros de ensaio porque ninguém viu a falha, o investimento se paga rápido.

Opção de evolução: Arduino UNO R4 WiFi

Na maquete, usamos 3 peças separadas que precisam conversar entre si por fios: o **Arduino Mega** (que controla a temperatura), o **ESP32** (que cuida do Wi-Fi e do Modbus) e o **módulo RTC DS3231** (que guarda a data e hora). São 3 placas, 3 alimentações, fios ligando uma na outra — mais pontos onde algo pode dar errado.

O **Arduino UNO R4 WiFi** resolve isso juntando tudo numa placa só:

- Tem um **processador Renesas** forte o bastante para rodar o PID (o controle de temperatura).
- Tem um **ESP32-S3 embutido** que já cuida do Wi-Fi e do Bluetooth — sem precisar de módulo separado.
- Tem **RTC interno** (relógio de tempo real) — sem precisar do módulo DS3231 externo.
- Tem **USB-C** moderno em vez do USB-B antigo do Mega.
- Se programa na mesma **IDE do Arduino** com a mesma linguagem — o código muda pouco.

Ou seja: **2 componentes viram 1** (Arduino + ESP32 viram uma placa só), e o RTC sai de graça. Menos fios, menos pontos de falha, menos espaço no painel.

Na fábrica, onde já existe Arduino, trocar o controlador atual por um UNO R4 WiFi significaria ganhar Wi-Fi, RTC e capacidade de ser o mestre Modbus RS-485 **sem adicionar nenhum módulo extra**. O custo de um UNO R4 WiFi fica em torno de **R$ 200 a R$ 280** — que é menos do que comprar um ESP32 industrial separado mais o RTC mais o conversor de nível lógico, e ainda sobra a economia de não precisar dos fios entre as placas.

Abordagem	Componentes	Custo estimado
Atual (maquete)	Arduino Mega + ESP32 + RTC + conversor de nível	~R$ 350 (4 peças + fios entre elas)
Evolução	**Arduino UNO R4 WiFi** (tudo em 1 placa)	~R$ 200 a R$ 280 (1 peça)

---

6. Quais são os riscos?

E se...	O que fazemos
Um componente atrasa para chegar?	O projeto é modular: dá para testar as outras partes enquanto espera.
O sensor erra a leitura?	Calibramos em bancada e colocamos filtros no software para ignorar leituras absurdas.
O dissipador esquenta demais?	Os coolers têm sensores de rotação. Se parar de girar, o sistema desliga a Peltier na hora.
O Wi-Fi cai?	O controle continua funcionando sozinho. O log fica gravado no cartão de memória. Quando o Wi-Fi volta, os dados são reenviados.

---

7. Qual é o diferencial?

Câmaras climatizadas comuns controlam só a temperatura. A nossa proposta vai além: **monitora cada aparelho individualmente**. Se um dos 50 dispositivos morre no meio do teste, o operador sabe na hora qual foi e em que momento — em vez de descobrir 8 horas depois, quando o teste já acabou.

Isso é **rastreabilidade**, e é o que transforma um ensaio genérico em um processo controlado e confiável.

---

8. Divisão da apresentação (5 integrantes, ~2 min cada)

Quem	Fala sobre	Frase-chave
Integrante 1	O problema da empresa e por que ele importa	"O operador não sabe qual aparelho falhou nem quando. Nosso projeto resolve isso."
Integrante 2	A parte elétrica: painel, emergência, proteções	"A segurança é em hardware. Se o programa travar, a máquina desliga sozinha."
Integrante 3	O controle de temperatura (PID) e os sensores	"O PID mede, compara e ajusta sozinho. E a corrente de cada aparelho é vigiada o tempo todo pelo RS-485."
Integrante 4	O software, a IHM, o Wi-Fi e os dados	"Tudo é registrado com data e hora. Se a rede cair, o sistema continua funcionando sozinho."
Integrante 5	Custos, viabilidade e escalabilidade industrial	"O protótipo custa menos de R$ 2.000. Na fábrica, com R$ 3.300 a R$ 5.400 monitora os 50 dispositivos."

---

9. O que todo mundo do grupo precisa saber responder

1. **Qual é o problema?** O teste térmico roda por horas e ninguém sabe se um aparelho parou de funcionar no meio.
2. **Qual é a solução?** Um painel com sensores de corrente que monitora cada posição individualmente e avisa na hora se algo falhar.
3. **É seguro?** Sim. O botão de emergência e as proteções funcionam em hardware, sem depender do programa.
4. **Quanto custa?** Menos de R$ 2.000 o protótipo. Na fábrica real, de R$ 3.300 a R$ 5.400 para monitorar os 50 dispositivos com módulos RS-485.
5. **Escala pra fábrica?** Sim. Usa 7 módulos de 8 canais ligados em RS-485, um ESP32 industrial na ponta e o dashboard é gratuito.
6. **O que é PID?** É o controle automático de temperatura: mede, compara com o desejado e ajusta. Igual o ar-condicionado de casa, só que feito por nós (ver seção 10).
7. **O que é RS-485?** É um barramento industrial de 2 fios que permite ligar vários módulos em série e ler todos com um único controlador (ver seção 11).
8. **Qual é o diferencial?** Saber **qual** aparelho falhou e **quando**, com registro automático — e não só controlar a temperatura.

---

10. O que é malha fechada e PID? (para a banca perguntar)

Imagine o ar-condicionado da sua casa. Você coloca em 22 °C e ele faz o resto sozinho: esfria quando está quente, desliga quando chega na temperatura, liga de novo se esquentar. Ele **mede** a temperatura, **compara** com o que você pediu e **age** para corrigir a diferença. Isso é uma **malha fechada** — o sistema se autoajusta.

Agora, o **PID** é a "receita" que o sistema usa para decidir **quanto** esfriar ou esquentar. As três letras significam:

- **P (Proporcional):** "Quanto mais longe eu estou da temperatura certa, mais forte eu ligo." Se falta muito para chegar em 22 °C, ele liga no máximo. Se falta pouco, liga devagar.
- **I (Integral):** "Se eu estou pertinho mas nunca chego exatamente, vou aumentando um pouquinho até chegar." O I corrige erros pequenos que o P sozinho não consegue zerar.
- **D (Derivativo):** "Se a temperatura está caindo rápido e vai passar do ponto, vou freando antes." O D evita que o sistema passe do valor desejado e fique oscilando.

Na prática do nosso projeto: O Arduino mede a temperatura da câmara a cada 1 segundo. Se está acima do desejado, liga a Peltier (frio). Se está abaixo, liga o PTC (calor). O PID calcula exatamente quanto de potência enviar (de 0% a 100%) para chegar na temperatura certa sem ficar ligando e desligando feito louco.

> **Por que isso importa pro curso?** Porque malha fechada e PID são a base de qualquer automação industrial: controle de nível de tanques, velocidade de motores, pressão de caldeiras. Aprender na câmara térmica serve pra qualquer processo.

---

11. O que é RS-485 e como funciona? (para a banca perguntar)

Imagine que você tem 50 sensores espalhados num quadro elétrico e quer ler todos eles com um único ESP32. Não dá para ligar 50 fios analógicos no ESP32 (ele não tem 50 entradas), e mesmo se tivesse, os fios longos pegariam interferência dos motores e dariam leituras erradas.

A solução é o **RS-485**: um barramento industrial que funciona com **apenas 2 fios** (chamados A e B).

Como funciona, passo a passo:

1. **Os sensores são agrupados:** A cada 8 sensores, tem um "módulo concentrador" (módulo ADC). Ele lê os 8 sensores que estão perto dele e guarda os valores na memória.
2. **Os módulos são ligados em corrente:** O fio A do primeiro módulo liga no A do segundo, que liga no A do terceiro... e assim por diante. São só 2 fios percorrendo todos os módulos, um atrás do outro (isso se chama **daisy-chain**, ou "corrente em margarida").
3. **O ESP32 pergunta um por um:** O ESP32 é o "mestre" da rede. Ele manda uma mensagem: "Módulo 1, me diz seus 8 valores". O módulo 1 responde. Depois pergunta pro módulo 2, pro 3, e assim por diante. Esse esquema de perguntar e responder se chama **Modbus RTU** — é um protocolo padrão usado no mundo inteiro em fábricas.
4. **Por que funciona bem em ambiente industrial?** Porque o RS-485 não manda tensão absoluta (tipo 0 V ou 5 V, que qualquer motor por perto bagunça). Ele manda a **diferença** de tensão entre os 2 fios. Se o ruído sobe nos dois fios ao mesmo tempo, a diferença fica igual — o ruído se cancela sozinho. Por isso o RS-485 funciona bem a **distâncias de até 1.200 metros**, mesmo perto de motores, compressores e inversores de frequência.
5. **Por que não usar sensores 4-20 mA para longas distâncias?**
   A indústria ama sensores 4-20 mA porque eles também andam longas distâncias sem perder o sinal. Mas se fôssemos usar transmissores 4-20 mA para 50 aparelhos, esbarraríamos em três problemas graves:

- **Custo e Cabos:** Seriam 50 transmissores muito caros e 50 cabos cruzando a fábrica até o painel.
- **Portas limitadas:** O Arduino Mega só tem 16 entradas analógicas. O ESP32 tem menos ainda. Não teríamos onde ligar 50 fios analógicos.
- **Leitura incompatível:** Microcontroladores só leem *Tensão* (0 a 5 Volts). Para ler Corrente (4-20 mA), precisaríamos instalar 50 resistores de precisão ou 50 plaquinhas conversoras no painel. No fim das contas, para o Arduino ler os 50 sensores 4-20 mA, teríamos que usar módulos concentradores RS-485 de qualquer jeito!
  Por isso nossa solução corta caminho: usamos sensores simples e baratos perto das lâmpadas, ligamos eles direto nos módulos RS-485 lá na câmara, e puxamos **um único cabo de 2 fios** cruzando a fábrica. Economizamos muito dinheiro e resolvemos a falta de portas do Arduino.

No nosso protótipo (maquete):

- Temos 2 sensores de corrente ligados num módulo ADC de 2 canais.
- O módulo ADC se comunica com o ESP32 pelo RS-485, usando Modbus.
- É a mesma coisa que a fábrica faria com 50 sensores e 7 módulos — só que em escala menor.

Por que isso importa pro curso? Porque RS-485 e Modbus são a base de comunicação de qualquer automação industrial: CLPs, inversores de frequência, medidores de energia, IHMs. Aprender a usar isso na maquete é aprender a linguagem que as máquinas da fábrica falam.

---

12. Perguntas "Pegadinha" que a banca pode fazer (e como responder)

Se a banca quiser testar se vocês realmente entendem de eletrotécnica e não só de programação, eles farão perguntas difíceis. Aqui estão as principais e como responder:

Pergunta 1: "Por que usar 24 Volts para alimentar as Peltier e o Aquecedor, e não 110V ou 220V direto da tomada?"
Como responder: "Por segurança e norma (NR-10). Como é uma maquete didática que alunos vão operar, optamos por trabalhar com **SELV** (*Safety Extra Low Voltage*, ou Extra-Baixa Tensão de Segurança). Uma rede SELV de 24V DC é eletricamente isolada da rede elétrica principal e tem uma tensão tão baixa que não causa choque letal caso alguém encoste nos fios acidentalmente. Na fábrica real, o comando continua em 24V, mas a força bruta seria 220V/380V."

Pergunta 2: "O que acontece se o Arduino travar enquanto o aquecedor estiver ligado no máximo? Vai pegar fogo?"
Como responder: "Não, porque a segurança é garantida no hardware. Usamos resistores de **pull-down**. (*Se a banca pedir para explicar:* 'Pull-down' significa 'puxar para baixo'. É um resistor que garante que, se o Arduino travar ou o fio de sinal se soltar, a tensão do comando caia instantaneamente para o GND / 0V. O seu oposto é o 'pull-up', que garante nível alto, 5V, quando inativo). Com o pull-down, a chave de potência zera e se auto-desliga se perder comunicação. Além disso, o Arduino tem um *Watchdog* que o reinicia se travar, e temos o botão de emergência que corta tudo fisicamente."

Pergunta 3: "Se isso é para a indústria, por que usar Arduino e não um CLP (Controlador Lógico Programável) de mercado?"
Como responder: "Para fins didáticos, o Arduino e o ESP32 nos forçam a aprender a lógica de baixo nível (como calcular um PID matematicamente e como montar a rede Modbus do zero), conhecimentos que o CLP esconde do operador. Além disso, nós documentamos no projeto que a lógica migra perfeitamente para CLPs baseados em Arduino que já são certificados para a indústria, como o *Arduino Opta* ou o *Portenta Machine Control*, aproveitando o mesmo código."

Pergunta 4: "Vocês estão colocando componentes de FORÇA (Peltier/PTC consumindo 10A) e de COMANDO (Arduino/ESP32 de 5V) no mesmo painel. Isso não vai causar ruído magnético e travar as placas?"
Como responder: "Poderia causar interferência eletromagnética (EMI), mas nós montamos o painel seguindo exatamente as práticas da indústria: nós **não misturamos os fios**. Separamos o layout físico: usamos canaletas exclusivas para os fios de Força (cabos grossos) e canaletas separadas para o Comando (fios finos de sinal). A Força fica embaixo, o Comando em cima. Além disso, as fontes de alimentação possuem filtros de isolamento e usamos o protocolo RS-485 para os sensores, que foi inventado exatamente para rejeitar ruídos eletromagnéticos do chão de fábrica."

---

13. Encerramento sugerido

"Concluímos que o projeto é viável porque usa componentes baratos e comerciais, a montagem é por etapas e a segurança não depende de software. Ele aplica na prática o que aprendemos em comandos elétricos, instalações, instrumentação e automação. E o mais importante: resolve um problema real da empresa — saber na hora quando um dispositivo falha, em vez de descobrir horas depois. Na fábrica, com um investimento de R$ 3.300 a R$ 5.400, o sistema monitora os 50 dispositivos usando a mesma arquitetura RS-485 que provamos funcionar na maquete."